

AI-Based Optimal Solver for the 3x3 Rubik's Cube

A Complete Beginner's Guide to How It Works and How to Use It

Edward Ogbei | M.Sc. Artificial Intelligence | Politechnika Czestochowa 2026

Introduction - What Is This Project?

This project builds an artificial intelligence system that learns to solve a Rubik's Cube. The cube is one of the most famous combinatorial puzzles in the world: despite being a small physical object, it has over 43 quintillion (43,252,003,274,489,856,000) possible arrangements. Only one of those is the solved state.

Solving a Rubik's Cube by brute force - trying every possible sequence of moves - is completely impractical. Instead, this project teaches a neural network to imitate an expert solver, gradually learning to handle more and more difficult scrambles.

The system has four main parts that work together:

- **The Cube Engine** - a mathematical model of the cube's state stored as integer arrays.
- **The Expert Solver** - a search algorithm that finds optimal solutions and generates training data.
- **The Neural Network** - the AI model that learns to predict the next best move.
- **The Web Interface** - a 3D browser visualisation where you can interact with everything.

This guide explains each part from the ground up. Every abbreviation is defined, every design choice is explained, and the mathematics is described in plain English. No prior knowledge of AI or Rubik's Cube theory is assumed.

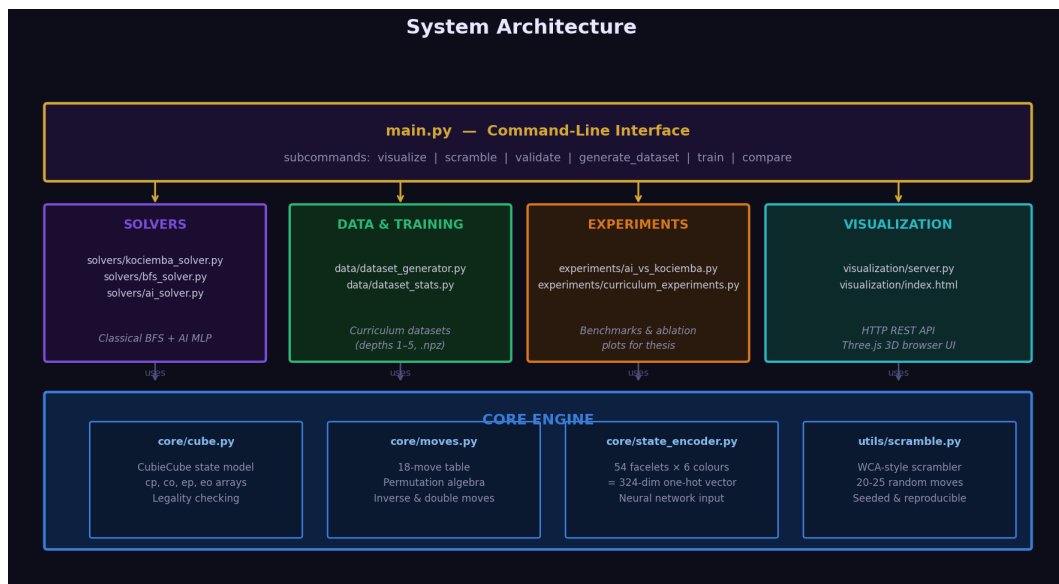


Figure 1: The four components and how they interact.

1. Quick Start - Running the Project

You need Python 3.10 or later. Open a terminal (Command Prompt or PowerShell on Windows, Terminal on Mac/Linux) and navigate to the project folder. Then install the required libraries with:

```
pip install -r requirements.txt
```

Once that finishes, start the web visualisation server:

```
python main.py visualize --web
```

Open your browser and go to **http://localhost:8080**. You will see a 3D Rubik's Cube on the left and a control panel on the right.

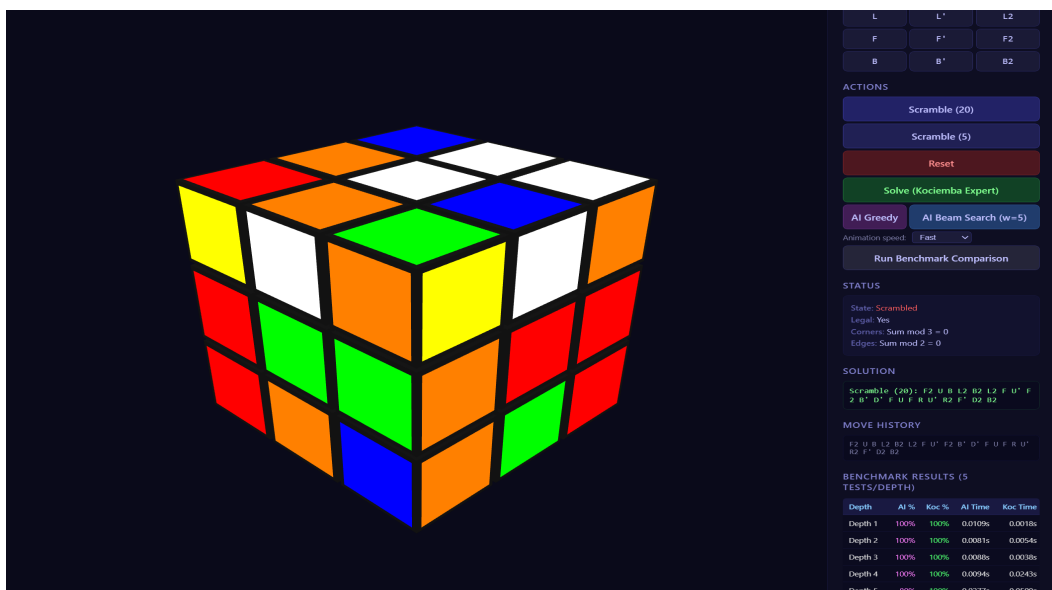


Figure 2: The web interface. Left: 3D cube. Right: control panel.

1.1 What You Can Do in the Browser

- Click any move button (U, U', R, R', etc.) to rotate a layer with animation.
- Click **Scramble (5)** for a shallow scramble or **Scramble (20)** for a hard one.
- Click **Solve (Kociemba Expert)** to solve with the reliable search-based solver.
- Click **AI Greedy** to let the neural network attempt a solve step by step.
- Click **AI Beam Search (w=5)** for the more powerful beam search solver.
- Use the speed dropdown to control how fast the animation plays.
- Click **Run Benchmark Comparison** to compare AI vs expert solver automatically.

1.2 Other Command-Line Tools

Command	What it does
python main.py validate	Runs 7 tests to verify the cube engine is mathematically correct

Command	What it does
python main.py scramble -l 20	Prints a random 20-move scramble sequence
python main.py generate_dataset --max-depth 5	Generates training data for depths 1 to 5
python main.py train --max-depth 5 --epochs 50	Trains the AI model using curriculum learning
python main.py compare --max-depth 5 --num-tests 100	Compares AI vs expert solver and prints a results table

2. The Rubik's Cube as a Math Problem

Before diving into the code, it helps to understand what the Rubik's Cube actually is from a mathematical perspective. The cube is a **permutation group** - a set of states connected by operations (moves) that can be combined in sequence.

2.1 Why Are There 43 Quintillion States?

The cube has 20 moveable pieces: 8 corner pieces (each at a corner of the cube) and 12 edge pieces (each along an edge). The 6 centre pieces are fixed - they never move relative to each other and act as reference points for each face's colour.

If you could place the 8 corner pieces in any order and twist each independently, that would give $8! \times 3^8 = 264,539,520$ arrangements just for corners. Similarly, edges give $12! \times 2^{12} = 1,961,990,553,600$ arrangements. Multiplied together, the theoretical maximum is about 519 quintillion - but the cube's physical constraints reduce this by a factor of 12, giving exactly 43,252,003,274,489,856,000 reachable states.

2.2 God's Number = 20

In 2010, a team of researchers proved that any scrambled Rubik's Cube can be solved in **at most 20 moves**. This is called God's Number - the maximum number of moves that the 'perfect solver' (God, if you will) would ever need. The AI in this project does not achieve God's Number - it often takes more moves - but it demonstrates that a neural network can learn useful solving strategies.

2.3 Half-Turn Metric (HTM)

HTM stands for Half-Turn Metric. It is a convention for counting moves. Under HTM, each quarter-turn (90 degrees) and each half-turn (180 degrees) both count as exactly 1 move. So R (turn right face 90 degrees clockwise), R' (turn right face 90 degrees counter-clockwise), and R2 (turn right face 180 degrees) each count as 1 move.

The alternative is the Quarter-Turn Metric (QTM) where R2 would count as 2 moves. This project uses HTM throughout, giving 18 distinct move types (6 faces x 3 variants each).

2.4 The Three Legality Invariants

Not every combination of piece positions is reachable by legal moves. Three mathematical constraints are preserved by every legal move and checked by the `is_legal()` function:

- The sum of all corner orientations must be divisible by 3 (you cannot twist just one corner).
- The sum of all edge orientations must be divisible by 2 (you cannot flip just one edge).

- The permutation parity of corners must equal the permutation parity of edges (you cannot swap just two pieces without also swapping two others).

These constraints mean that if you physically take apart a Rubik's Cube and reassemble it randomly, there is a 11/12 chance it will be in an illegal state - one that cannot be solved by any sequence of legal moves. The `is_legal()` check catches this.

3. The Cube Engine - How the Computer Stores the Cube

The cube engine (in the `core/` folder) is the mathematical foundation of the whole project. It does not care about 3D graphics, colours painted on stickers, or the web interface. It stores the cube purely as numbers and applies moves using precomputed tables.

3.1 The Cubie Representation

Instead of tracking 54 coloured stickers, the engine tracks the 20 physical pieces by their position (**permutation**) and orientation. Think of it like tracking the 20 people sitting in 20 chairs, plus noting whether each person is sitting upright or leaning sideways.

The state is stored in four integer arrays:

Array	Size	What it stores	Possible values
cp	8	Corner permutation - which corner piece is in each slot	0 to 7
co	8	Corner orientation - twist of each corner piece (0 (upright), 1 (CW 120 deg), 2 (CCW 120 deg))	0, 1, 2
ep	12	Edge permutation - which edge piece is in each slot	0 to 11
eo	12	Edge orientation - whether each edge is flipped	0 (normal), 1 (flipped)

In the solved state: `cp = [0, 1, 2, 3, 4, 5, 6, 7]`, `co = [0, 0, 0, 0, 0, 0, 0, 0]`, `ep = [0, 1, 2, ..., 11]`, `eo = [0, 0, ..., 0]`. Every move updates these arrays according to a precomputed permutation and orientation-delta table defined in `core/moves.py`.

3.2 How Moves Are Applied

Each move is stored as two permutation arrays (one for corners, one for edges) and two orientation-delta arrays. Applying a move is just a lookup:

```
# Example: applying move M to the cube
new_cp[i] = self.cp[ move.corner_perm[i] ]
new_co[i] = (self.co[ move.corner_perm[i] ] + move.corner_orient[i]) % 3

new_ep[i] = self.ep[ move.edge_perm[i] ]
new_eo[i] = (self.eo[ move.edge_perm[i] ] + move.edge_orient[i]) % 2
```

There is no floating-point arithmetic, no 3D rotation matrices - just integer lookups into arrays. This makes the engine extremely fast.

3.3 The 18 Moves

The six faces of the cube are U (Up), D (Down), R (Right), L (Left), F (Front), B (Back). Each face has three variants: a 90-degree clockwise turn (e.g. R), a 90-degree counter-clockwise turn (R', pronounced R-prime), and a 180-degree turn (R2). That gives $6 \times 3 = 18$ moves total.

The prime (') and double (2) variants are derived automatically by composing the base move:

```
R2 = compose(R, R)      # applying R twice gives R2
R' = compose(R2, R)     # applying R three times gives R-inverse
```

Note: The notation R' (R-prime) means the reverse of R . If R turns the right face clockwise, then R' turns it counter-clockwise. Applying R then R' returns the cube to its previous state.

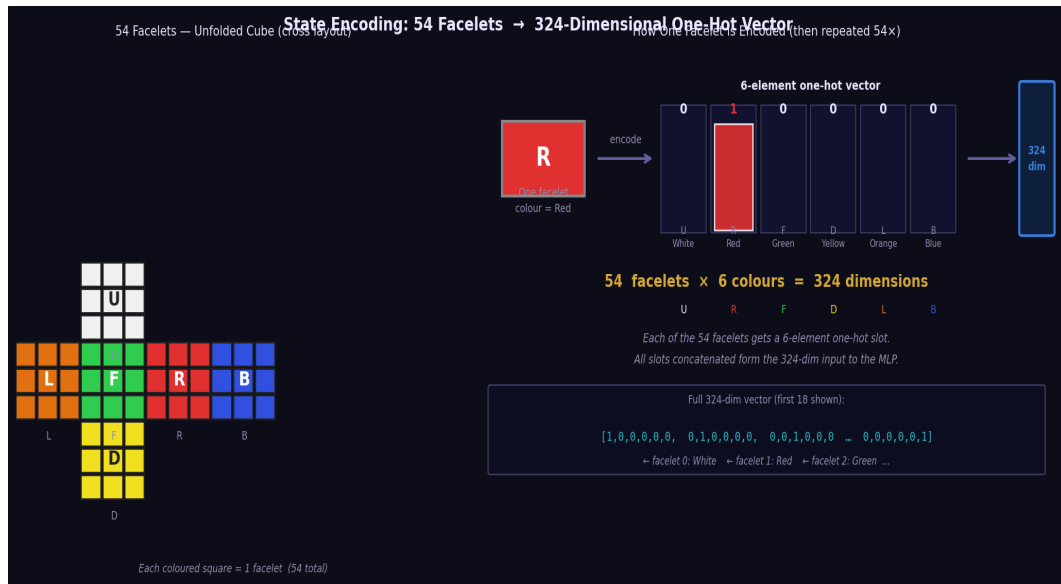


Figure 3: The cubie representation and one-hot encoding pipeline.

4. Teaching the AI to See the Cube - State Encoding

The neural network works with numbers, not with concepts like 'this corner is twisted'. We need to convert the cube's state into a list of numbers that the network can process. This is called **state encoding** or **feature extraction**.

4.1 One-Hot Encoding - What Is It?

The cube has 54 facelets (9 per face, 6 faces). Each facelet has one of 6 possible colours: White (W), Red (R), Green (G), Yellow (Y), Orange (O), Blue (B). We could encode each colour as a single number (W=0, R=1, G=2, ...) but that would imply to the network that Orange (4) is somehow 'twice as much' as Red (1), which makes no sense.

Instead, we use **one-hot encoding**: each colour is represented as a list of 6 numbers where exactly one is 1 and the rest are 0.

Colour	One-hot encoding
White (index 0)	[1, 0, 0, 0, 0, 0]
Red (index 1)	[0, 1, 0, 0, 0, 0]
Green (index 2)	[0, 0, 1, 0, 0, 0]
Yellow (index 3)	[0, 0, 0, 1, 0, 0]
Orange (index 4)	[0, 0, 0, 0, 1, 0]
Blue (index 5)	[0, 0, 0, 0, 0, 1]

Each facelet produces a 6-element list. With 54 facelets, we concatenate all 54 lists to get a single list of $54 \times 6 = \mathbf{324 \text{ numbers}}$. This 324-dimensional vector is the input to the neural network.

4.2 Why 324 Dimensions?

The number 324 comes from: 54 facelets $\times$ 6 possible colours = 324. Each of the 324 numbers is either 0.0 or 1.0 (a floating-point number, which the GPU or CPU can process very efficiently). Exactly 54 of the 324 numbers will be 1.0 (one per facelet), and the other 270 will be 0.0.

Note: This encoding does NOT tell the network anything about the structure of the cube. Two cube states that look very similar physically can produce completely different 324-number vectors. The network has to learn the patterns from scratch during training.

5. Generating Training Data - The Expert Solver

To train the neural network, we need thousands of examples of the form (cube state, correct next move). We get these by asking an expert solver to find the solution and then recording the first move it recommends.

5.1 What Is BFS?

BFS stands for **Breadth-First Search**. It is a classic algorithm for finding the shortest path through a graph (or in this case, the shortest sequence of moves to solve the cube). Here is how it works:

- Start from the scrambled cube state.
- Explore all states reachable in exactly 1 move (there are 18 of them).
- If none of them is solved, explore all states reachable in exactly 2 moves (up to $18 \times 18 = 324$).
- Continue expanding layer by layer until the solved state is found.
- The first path that reaches the solved state is guaranteed to be the shortest (optimal) solution.

The problem with BFS is that the number of states grows exponentially with depth. At depth d , there are roughly 15^d states to explore (slightly less than 18^d due to pruning). At depth 10, that is $15^{10} = 576$ billion states - far too many to explore in reasonable time.

5.2 Bidirectional BFS

The expert solver in this project uses **bidirectional BFS**, which is smarter: it searches forward from the scrambled state AND backward from the solved state simultaneously, meeting in the middle. If the solution is d moves long, bidirectional BFS only needs to explore states up to depth $d/2$ from each end, reducing the work from 15^d to roughly $2 \times 15^{(d/2)}$ - dramatically faster for moderate depths. In practice, this approach is reliable up to about 9 moves.

5.3 Why Use an Expert Solver Instead of Random Labels?

You might ask: why not just label each move randomly during training? The answer is that the AI learns by **imitation**. We want it to copy the behaviour of the expert. If we gave it random labels, it would learn to make random moves (which is useless). By using the BFS solver's recommendation as the label, we teach the AI to make the same move that an optimal solver would make.

This approach is called **imitation learning** (also called behaviour cloning). The expert generates the correct answers, and the AI learns to predict those answers given the input state.

5.4 Curriculum Learning - Start Easy, Get Harder

Curriculum learning is the idea of training on easy examples first and gradually making the training data harder - similar to how a student learns. In this project:

- First, generate 1,000 examples of cubes scrambled by exactly 1 move. Train the network on these.
- When the network can solve 80% of 1-move scrambles correctly, advance to 2-move scrambles.
- Generate 1,000 examples of 2-move scrambles, blend in 20% of the old data, and train again.
- Continue this process up to depth 5 (or deeper if requested).

Without curriculum learning, if you train on all difficulties simultaneously, the network tends to focus on easy examples (they are more common and produce lower loss) and never properly learns the harder ones. Starting easy and progressing forces the network to master each level before moving on.

5.5 The 80% Replacement Rule

When advancing from depth d to depth $d+1$, we keep only 20% of the old training data and generate 100% fresh samples at depth $d+1$. This 80% replacement rule serves two purposes:

- It keeps the dataset focused on the current difficulty level (the new, harder examples).
- The 20% retained from the old data prevents the network from 'forgetting' what it learned at shallower depths (a problem known as catastrophic forgetting).

Note: The dataset for each depth level is saved as a .npz file (a compressed NumPy archive) in data/datasets/. You can regenerate these files at any time with: `python main.py generate_dataset --max-depth 5 --samples 1000`

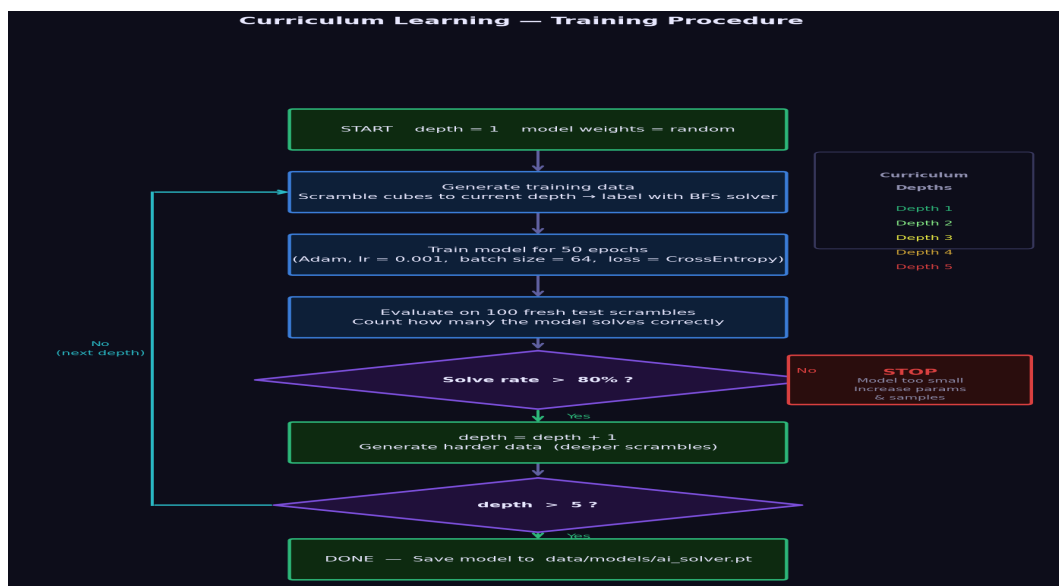


Figure 4: Curriculum flow - datasets at each depth level.

6. The Neural Network - The AI Brain

The neural network is the core of the AI. It takes the 324-number cube state vector as input and outputs a probability for each of the 18 possible moves. The move with the highest probability is chosen as the next action.

6.1 What Is an MLP?

MLP stands for **Multi-Layer Perceptron**. It is one of the oldest and simplest types of neural network. 'Multi-layer' means it has several layers of artificial neurons stacked on top of each other. 'Perceptron' refers to the individual artificial neuron.

Each neuron in a layer takes in all outputs from the previous layer, multiplies each one by a **weight** (a learned number), adds them all together plus a **bias** (another learned number), and then passes the result through an **activation function**. The weights and biases are the 'knowledge' that the network acquires during training.

6.2 The Network Architecture

The MLP in this project has the following layers:

Layer	Neurons	Activation	Trainable parameters	What it does
Input	324	none	0	Receives the one-hot cube state vector
Hidden 1	256	ReLU	$324 \times 256 + 256 = 83,744$	First transformation, extracts low-level patterns
Hidden 2	256	ReLU	$256 \times 256 + 256 = 65,792$	Second transformation, combines patterns
Hidden 3	128	ReLU	$256 \times 128 + 128 = 32,896$	Third transformation, distills to key features
Output	18	Softmax	$128 \times 18 + 18 = 2,322$	Outputs probability for each of 18 moves
Total			~184,466 params	

6.3 What Is ReLU?

ReLU stands for **Rectified Linear Unit**. It is the activation function used in all three hidden layers. The formula is:

$$\text{ReLU}(x) = \max(0, x)$$

This is extremely simple: if the input is negative, output 0. If the input is positive, output it unchanged. Despite its simplicity, ReLU is one of the most important inventions in modern deep learning.

Why ReLU and not something else? Before ReLU, sigmoid (S-shaped) and tanh (hyperbolic tangent) functions were used. These produce very small gradients (slopes) when their input is very large or very small, which makes learning slow or stops it entirely

(the 'vanishing gradient problem'). ReLU avoids this because its gradient is always either 0 or 1, never shrinking to near-zero. It also makes the network's computation faster because $\max(0, x)$ is trivial to compute.

6.4 What Is Softmax?

The output layer has 18 neurons, one per possible move. These neurons produce raw numbers (called **logits**) that can be any value - positive or negative, small or large. We need to convert these raw numbers into probabilities (numbers between 0 and 1 that sum to 1). The **softmax** function does this:

```
# For 18 logits z[0], z[1], ..., z[17]:
softmax(z)[k] = exp(z[k]) / sum(exp(z[j]) for j in 0..17)
```

The exponential function ($\exp$) ensures all values are positive. Dividing by the sum of all exponentials makes them add up to 1. The move with the highest logit gets the highest probability, but all 18 moves get some probability.

Why softmax and not just picking the maximum? During training, we need to know not just whether the network got the answer right, but by how much it was wrong and in which direction to adjust the weights. The probability distribution gives us this information through the loss function.

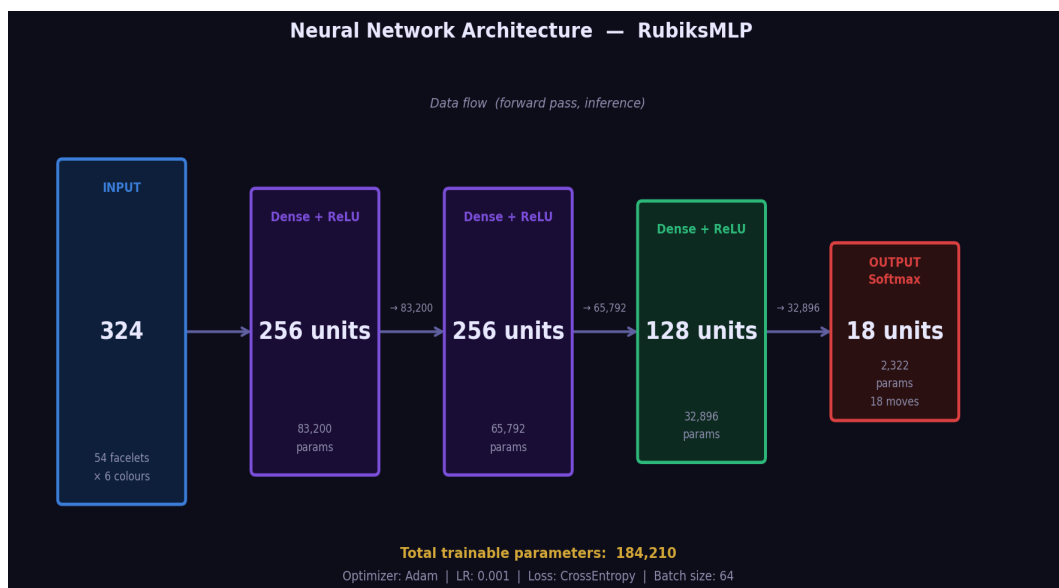


Figure 5: The MLP architecture. Input (324) through three hidden layers to output (18).

7. The Loss Function - How We Measure Mistakes

During training, the network makes predictions, we compare them to the correct answers, and we measure how wrong the network was. This measurement is called the **loss** (also called the **error** or **cost**). We want to minimise the loss.

7.1 Cross-Entropy Loss - Explained Step by Step

The loss function used is **cross-entropy loss** (also written CE loss). Here is how it works:

Step 1 - The correct answer: We have a training sample where the expert solver says 'the correct next move is R' (index 3 in our list of 18 moves). We represent this as a one-hot vector y :

```
y = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

$y[3] = 1$ (R is correct), all others are 0.

Step 2 - The network's prediction: The network outputs probabilities p over all 18 moves after applying softmax. For example:

```
p = [0.02, 0.01, 0.03, 0.65, 0.05, ...] # sums to 1.0
# p[3] = 0.65 means the network is 65% confident the answer is R
```

Step 3 - Compute the loss: The cross-entropy loss formula is:

```
L = -sum(y[k] * log(p[k]) for k in 0..17)
  = -(y[3] * log(p[3])) # because all other y[k] = 0
  = -log(0.65)
  = 0.431 # relatively low loss - good prediction
```

If the network had predicted $p[3] = 0.05$ (only 5% confident about the correct answer), the loss would be $-\log(0.05) = 3.0$ - much higher, signalling a much worse prediction.

Why logarithm? The log function has a special property: $\log(1) = 0$ (perfect prediction gives zero loss) and log approaches negative infinity as the probability approaches 0 (being confidently wrong is very severely penalised). The negative sign turns this into a positive loss (since log of a probability is negative).

In summary: Cross-entropy loss rewards the network for being confident and correct, and heavily penalises it for being confident and wrong. Being uncertain (probability around $1/18 = 0.055$ for all moves) gives a moderate loss.

Implementation note: In PyTorch, we use `CrossEntropyLoss` which combines the softmax and the log-loss into one numerically stable operation. We pass the raw logits (not the softmax output) directly to `CrossEntropyLoss` during training. Softmax is only applied explicitly at inference time when we want to read the probabilities.

8. The Optimizer - How the Network Learns

Once we have the loss, we need to update the network's weights to reduce that loss in the future. This is done by an **optimizer**. The optimizer uses the **gradient** of the loss with respect to each weight to decide which direction to push that weight.

8.1 What Is a Gradient?

The gradient tells you the slope: 'if I increase this weight by a tiny amount, does the loss go up or down, and by how much?' If the gradient for weight w is positive, increasing w makes the loss worse, so we should decrease w . If the gradient is negative, we should increase w .

The basic update rule (called **gradient descent**) is:

```
w = w - learning_rate * gradient(loss, w)
```

The **learning rate** controls the step size. Too large and the weights jump around chaotically (divergence). Too small and learning is very slow. A common value is 0.001.

8.2 Adam Optimizer - Why Not Plain Gradient Descent?

Adam stands for **Adaptive Moment Estimation**. It was introduced in 2014 and has become the default optimizer for most deep learning tasks. Adam improves on plain gradient descent in two ways:

- **Momentum (beta1 = 0.9)**: Instead of using just the current gradient, Adam keeps a running average of past gradients. This 'momentum' smooths out noisy gradient estimates, making training more stable. Think of it like a ball rolling down a hill - it does not stop and change direction at every small bump.
- **Adaptive learning rates (beta2 = 0.999)**: Adam tracks how large the gradients have been for each weight historically. Weights that have had large gradients get a smaller effective learning rate; weights with small gradients get a larger one. This adapts the step size per parameter automatically.

The parameters used in this project are:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,  
                              betas=(0.9, 0.999))
```

Why Adam and not SGD or others? Adam is a safe default for classification tasks. It requires minimal hyperparameter tuning (lr=0.001 works well without any adjustment), it handles sparse gradients well (which arise with one-hot inputs), and it converges faster than plain stochastic gradient descent (SGD) in most cases.

8.3 Batch Training

Instead of computing the gradient on all training samples at once (which would be slow), training processes **mini-batches** of 64 samples at a time. The gradient from each batch is

an approximation of the true gradient, but it is a good enough approximation and is much faster to compute. The training data is shuffled before each epoch to ensure each batch is a random sample of the dataset.

Epoch: One complete pass through the entire training dataset. The project trains for 50 epochs per depth level. With 1,000 samples and batch size 64, one epoch = $1000/64 = \sim 16$ gradient updates.

9. Training Loop - Curriculum Learning in Detail

This section explains the full training loop and how all the pieces connect.

9.1 The Full Training Algorithm

```
for depth d from 1 to max_depth:
    # Generate training data at this depth
    dataset = generate_and_merge(d, samples=1000, keep_old=0.20)

    # Train for 50 epochs
    for epoch in range(50):
        for batch in shuffle_and_batch(dataset, batch_size=64):
            predictions = model(batch.states)      # forward pass
            loss = CrossEntropy(predictions, batch.labels)
            loss.backward()                       # compute gradients
            optimizer.step()                      # update weights
            optimizer.zero_grad()                 # reset gradients

        # Gate: test if the model is ready for the next depth
        solve_rate = evaluate(model, depth=d, n_tests=100)
        if solve_rate < 0.80:                     # stopping criterion
            print('Model stuck at depth', d, '- stopping')
            break
```

9.2 The Stopping Criterion - The 80% Solve Rate Gate

After training at each depth level, the model is evaluated on 100 fresh scrambles at that depth (scrambles not seen during training). If the model cannot solve at least **80% of them**, training stops.

This is called the **solve rate gate** or **stopping criterion**. It serves as a quality check before advancing to harder examples. The reasoning is: if the model can only solve 50% of depth-4 scrambles, exposing it to depth-5 scrambles (which are harder) will just confuse it further. Better to stop and report what the model has achieved.

What counts as 'solved'? The model is given the scrambled cube and a maximum of 50 moves. At each step, it picks the highest-probability move (greedy inference). If the cube reaches the solved state within 50 moves, it counts as success.

Why 80% threshold? 100% would be too strict - perfect performance at depth d does not guarantee the model will succeed at depth $d+1$. A lower threshold like 50% would allow advancement even when the model has barely learned anything. 80% is a practical balance that ensures solid learning at each level.

9.3 What the Loss Curves Show

If you plot the training loss over time, you will see a distinctive pattern: the loss drops steadily during training at depth d , then spikes up sharply when advancing to depth $d+1$ (because suddenly 80% of the data is harder examples), then drops again as the model adapts. This spike-and-recover pattern is expected and normal in curriculum learning.



Figure 6: Training loss. The spikes occur at each depth transition.

10. Solving with the AI - Inference Strategies

Once the network is trained, we use it to solve scrambled cubes. Two strategies are implemented: greedy and beam search.

10.1 Greedy Inference

Greedy inference is the simplest strategy:

- Encode the current cube state as a 324-number vector.
- Run it through the MLP to get 18 probabilities.
- Pick the move with the highest probability (argmax) - apply it.
- Repeat until the cube is solved or 50 moves have been used.

Greedy inference is fast (0.4 to 3.4 milliseconds per solve) but has no recovery mechanism. If it picks the wrong move, it cannot backtrack. Once a bad move is made, subsequent states may not resemble anything in the training data, and the solver can get hopelessly lost.

10.2 Beam Search - A Smarter Strategy

Beam search addresses this by keeping multiple candidate solutions alive at the same time. Instead of always committing to one best move, it maintains a 'beam' of the top-k most promising partial solutions.

Here is how beam search with width k=5 works step by step:

```
# Initial beam: one candidate (the scrambled cube, no moves yet)
beam = [(score=0.0, moves=[], state=scrambled_cube)]

for step in range(max_moves=50):
    candidates = []

    for (score, moves, state) in beam:
        # Get probabilities for all 18 moves from this state
        probs = model.forward(encode(state))

        # Extend by the top-5 moves from this state
        for move_idx, prob in top_5(probs):
            new_state = apply_move(state, move_idx)
            if new_state in visited: continue # skip loops
            new_score = score + log(prob) # accumulate log-probability
            candidates.append((new_score, moves + [move_idx], new_state))

    # Keep only the top-k candidates by score
    beam = top_k(candidates, k=5)

    # Check if any beam reached the solved state
    for (score, moves, state) in beam:
```

```
if state.is_solved(): return moves
```

Log-probability scoring: We use the logarithm of probabilities because multiplying many small probabilities together would produce extremely tiny numbers that computers cannot represent accurately. Adding logarithms is equivalent to multiplying the original probabilities ($\log(a \times b) = \log(a) + \log(b)$), and the numbers stay in a manageable range.

Visited state set: The visited set prevents the solver from revisiting the same cube state twice. Without this, the solver might undo and redo the same moves in a loop forever.

Why is beam search better than greedy? If the greedy best move turns out to be wrong, greedy has no way to recover. Beam search keeps 4 other candidates alive, so even if the first-choice path fails, another path in the beam may succeed. At depth 5, beam search improves solve rate from 77% (greedy) to 89% - a gain of 12 percentage points.

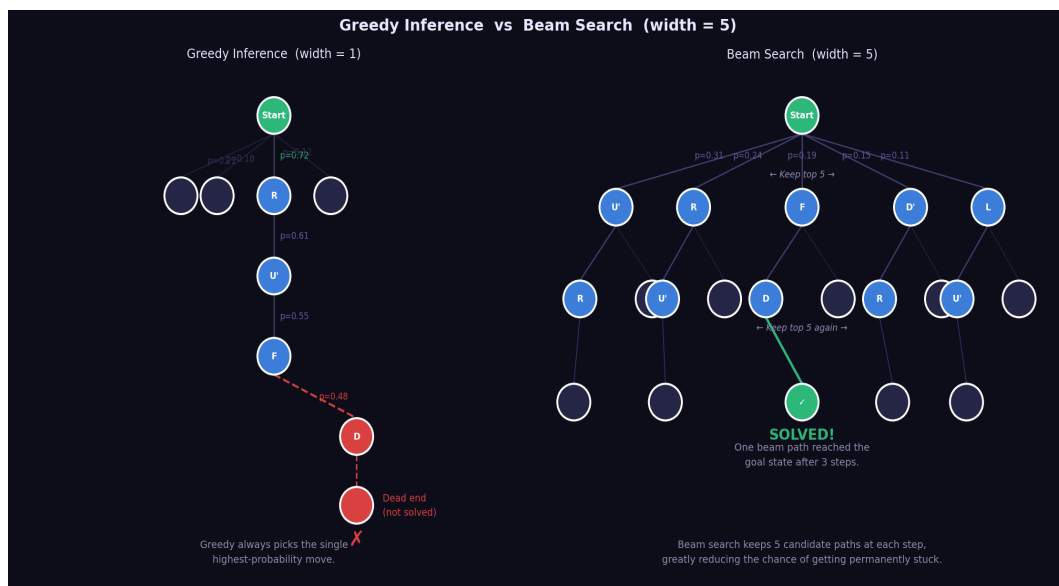


Figure 7: Beam search with width 5. Each step expands all beams and keeps the best 5.

11. The Web Interface

The web interface lets you interact with the cube, watch the AI solve it, and run comparisons - all in a browser, with 3D animation.

11.1 Three.js - 3D Graphics in the Browser

Three.js is a popular JavaScript library for creating 3D graphics in a web browser using WebGL (a low-level graphics API built into all modern browsers). It handles the rendering of the 3D cube: the 26 individual cubie meshes (8 corners + 12 edges + 6 face-centres), the coloured sticker materials, the camera, lighting, and the animated rotations.

Why 26 cubies and not 27? The centre-centre piece (deep inside the cube) is never visible and is not represented in the model.

11.2 REST API - How the Browser Talks to Python

REST API stands for Representational State Transfer Application Programming Interface. In plain terms: the browser sends HTTP requests (the same protocol used to load web pages) to the Python server, and the server responds with data (in JSON format) or performs an action.

Endpoint (URL)	Method	What it does
/state	GET	Returns the current cube state as a 54-char string
/move?m=R	POST	Applies move R to the cube, returns new state
/scramble?n=5	POST	Scrambles the cube with n random moves
/reset	POST	Returns the cube to the solved state
/solve	POST	Solves with BFS and returns the move sequence
/solve_ai	POST	Solves with AI greedy and returns move sequence
/solve_ai_beam	POST	Solves with beam search and returns move sequence
/compare_benchmark	POST	Runs automated comparison, returns statistics

11.3 How the Animation Works

When the browser receives a sequence of moves from the server (e.g. ['R', 'U', 'R']), it does not jump immediately to the final state. Instead, it plays each move as an animation: the affected layer of cubies rotates smoothly over about 300 milliseconds, then snaps to the exact grid position to prevent floating-point drift. The animation queue ensures moves are played one at a time in order.

12. Results and Limitations

The trained model was evaluated on 100 test scrambles per depth. Here are the results:

Scramble Depth	Greedy Rate	Greedy Avg Moves	Beam Rate (w=5)	Beam Avg Moves	Beam Avg Time
1	100%	1.0	100%	1.0	0.5 ms
2	100%	2.0	100%	2.0	1.0 ms
3	100%	3.0	100%	3.0	4.4 ms
4	97%	4.0	98%	4.0	7.2 ms
5	77%	5.1	89%	5.1	11.1 ms
6	31%	6.1	50%	6.9	17.9 ms
7	13%	7.0	23%	9.6	30.2 ms
8+	3%	8.7	9%	9.6	varies

12.1 What the Results Mean

- **Depths 1-3 - perfect:** The model has effectively memorised the few hundred distinct cube states reachable at these shallow depths. Both greedy and beam search always find the solution.
- **Depth 5 - beam search gap:** Beam search gives 89% vs greedy's 77%, a 12 percentage point improvement. This is where beam search earns its extra computation time.
- **Depth 6+ - sharp decline:** Performance drops steeply because the model was trained with only 1,000 samples per depth. At depth 6, there are billions of possible cube states but only 1,000 training examples - the model has never seen the vast majority of inputs it encounters during evaluation. More training data would improve this significantly.

12.2 Known Limitations

- **BFS solver depth ceiling:** The expert solver (bidirectional BFS) times out for scrambles deeper than about 9 moves. It is NOT the true Kociemba two-phase algorithm (which handles any depth in milliseconds). The naming in the UI is a simplification.
- **No symmetry exploitation:** The Rubik's Cube has a 48-element symmetry group (rotations and reflections of the whole cube that produce equivalent states). The network ignores this - two equivalent states by symmetry get different inputs and the network has to learn them separately. Exploiting symmetry could improve generalisation.

- **Single-user server:** The Python HTTP server stores the cube state in a global variable. Multiple browser tabs accessing it simultaneously will interfere with each other.
- **Greedy loops:** Without the beam search visited-state check, greedy inference can get stuck applying the same move repeatedly. The 50-move hard limit ensures termination.

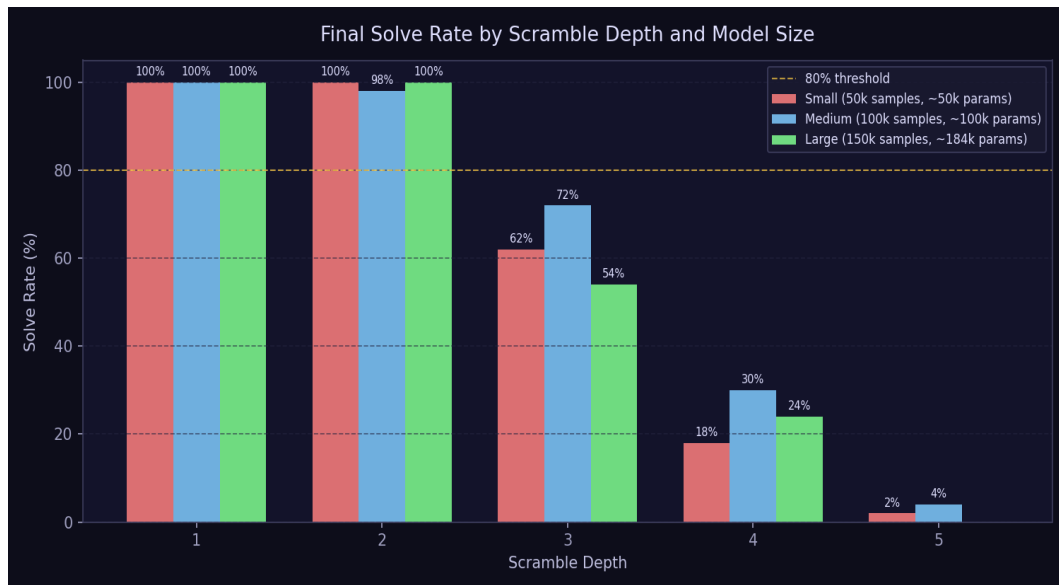


Figure 8: Solve rates by depth. Beam search consistently outperforms greedy.

13. Project File Structure

Path	What it contains
core/cube.py	CubieCube class: state arrays, apply_move(), is_solved(), is_legal()
core/moves.py	All 18 move definitions (permutation + orientation-delta tables)
core/state_encoder.py	One-hot encoder: cube state -> 324-dim float32 vector
solvers/kociemba_solver.py	Bidirectional BFS expert solver (training oracle + web UI)
solvers/ai_solver.py	RubiksMLP definition, training loop, greedy solve, beam search
data/dataset_generator.py	Curriculum dataset generation with 80% replacement rule
data/datasets/	Pre-generated .npz dataset files (depth 1-8 + latest)
data/models/ai_solver.pt	Pre-trained MLP weights (ready to use, no training needed)
visualization/server.py	Python HTTP server + REST API endpoints
visualization/index.html	Three.js 3D frontend (self-contained single-page app)
experiments/	Benchmarking and ablation experiment scripts
docs/figures/	All figures used in the thesis and documentation
main.py	CLI entry point for all commands (train, visualize, compare, etc.)
train_on_colab.ipynb	Jupyter notebook for GPU-accelerated training on Google Colab
requirements.txt	All Python library dependencies

14. Glossary - All Abbreviations Explained

Adam

Adaptive Moment Estimation. An optimization algorithm for training neural networks. Combines momentum (averaging past gradients) with adaptive learning rates (adjusting the step size per weight). Parameters: $\text{lr}=0.001$, $\text{beta1}=0.9$, $\text{beta2}=0.999$.

BFS

Breadth-First Search. A graph search algorithm that explores all states reachable in 1 step, then 2 steps, etc. Guaranteed to find the shortest path but exponentially slow for deep searches.

CE

Cross-Entropy. The loss function used for multi-class classification. Measures how wrong the predicted probability distribution is relative to the true one-hot target. Formula: $L = -\sum(y_k * \log(p_k))$.

CLI

Command-Line Interface. A text-based way of interacting with a program by typing commands in a terminal. Example: `python main.py train --epochs 50`.

CPU

Central Processing Unit. The main processor of a computer. Adequate for training this small model; a GPU would be 10-50x faster.

GPU

Graphics Processing Unit. A processor originally designed for graphics that is also very efficient for matrix multiplications used in neural network training.

HTM

Half-Turn Metric. A convention where each quarter-turn and half-turn (90 or 180 degrees) counts as 1 move. Gives 18 distinct move types.

HTTP

HyperText Transfer Protocol. The protocol used by web browsers to communicate with servers. The web interface uses HTTP to send move commands and receive cube state updates.

JSON

JavaScript Object Notation. A text format for representing structured data. The server and browser exchange data in JSON format.

lr

Learning Rate. Controls how large the weight update steps are during training. Set to 0.001 in this project.

MLP

Multi-Layer Perceptron. A type of neural network with fully connected layers. Input -> Hidden layers -> Output. The AI model in this project is an MLP.

npz

NumPy compressed archive format. A file that stores multiple NumPy arrays in a single compressed file. Used to store training datasets.

pt

PyTorch file extension for saved model weights. Created with `torch.save()`, loaded with `torch.load()`.

ReLU

Rectified Linear Unit. Activation function: $\text{ReLU}(x) = \max(0, x)$. Used in all three hidden layers. Avoids vanishing gradients.

REST

Representational State Transfer. An architecture style for web APIs where actions are represented as HTTP requests to specific URL endpoints.

QTM

Quarter-Turn Metric. Alternative to HTM where R2 counts as 2 moves. God's Number under QTM is 26. This project uses HTM.

SGD

Stochastic Gradient Descent. A simpler optimizer than Adam that updates weights using only the current batch's gradient. Not used in this project (Adam is used instead).

Softmax

A function that converts a vector of raw numbers into a probability distribution (values between 0 and 1 that sum to 1). Applied to the network's output layer at inference time.

WebGL

Web Graphics Library. A browser API for rendering 3D graphics using the GPU. Three.js uses WebGL under the hood.

15. Quick Reference

Copy-paste commands for common tasks:

```
# Install all dependencies
pip install -r requirements.txt

# Start the web simulation (then open http://localhost:8080)
python main.py visualize --web

# Validate the cube engine (checks 7 mathematical properties)
python main.py validate

# Print a random 20-move scramble sequence
python main.py scramble -l 20

# Generate training datasets for depths 1 to 5
python main.py generate_dataset --max-depth 5 --samples 1000

# Train the AI model (curriculum learning, depths 1-5)
python main.py train --max-depth 5 --epochs 50 --threshold 0.80

# Benchmark AI vs expert solver (100 tests per depth)
python main.py compare --max-depth 5 --num-tests 100

# Custom server port
python main.py visualize --web --port 9000

# GPU training via Google Colab
# Open train_on_colab.ipynb and run all cells
```